



Université Abou Bakr belkaid
faculté des sciences
département d'informatique

COURS GÉNIE LOGICIEL

NIVEAU : L3 INFORMATIQUE/3 EME INGÉNIEUR

Chargé de cours : S Meziane Tani

E –mail : s.mezianetani13@gmail.com

Octobre 2025

La modélisation



En quoi consiste la modélisation?

➤ Construire une représentation **abstraite** de la réalité

- ✓ Abstraction :
 - Ignorer les détails insignifiants.
 - Ressortir les détails les plus important .

Pourquoi modéliser?

❖ Pour gérer la complexité

- ❖ Des logiciels énormes Exemple :
- ❖ Windows 2000: 40 millions de ligne de code.

❖ Pour communiquer

- ❖ Plusieurs acteurs dans le cycle de développement
- ❖ Le code n'est pas toujours compréhensible par les développeurs qui ne l'ont pas écrit.

❖ Pour pérenniser un savoir-faire

- ❖ Certains projets peuvent durer des années
- ❖ Pas toujours les mêmes personnes qui travaillent sur le projet.

❖ Augmenter la productivité

- ❖ Génération de code à partir de modèles
- 

Quel langage utiliser ?

Plusieurs langages existaient /existent.

En réalité un seul a réussi à s'imposer comme langage pour la modélisation Orienté Objet .

UML



Chapitre 2 : UML : Unified Modeling Language



UML : Unified Modeling Language



Langage :

- **Syntaxe** et règles d'écriture
- Notations graphiques **normalisées**

... de modélisation :

- **Abstraction** du fonctionnement et de la structure du système
- **Spécification et conception**

... unifié :

- Fusion de plusieurs notations antérieures : Booch, OMT, OOSE
- Standard défini par l'OMG (Object Management Group)
- Dernière version : UML 2.5.1(2017)

En résumé : Langage graphique pour visualiser, spécifier, construire et documenter un logiciel

UML - Historique

- ❑ **1991** : première édition de Modélisation et conception orientées objet basée sur OMT, Object Modeling Technique, issue de la R&D de General Electric.
- ❑ **1994** : James Rumbaugh rejoint Rational et travaille avec Grady Booch à la fusion des notations OMT et Booch.
- ❑ **1995** : Ivar Jacobson rejoint Rational et intègre Objectory au travail d'unification.
- ❑ **1997** : l'OMG (Object Management Group) accepte UML, proposé par Rational, comme standard de modélisation objet.
- ❑ **2001** : révision par l'OMG d'UML 1.
- ❑ **2004** : adoption d'UML 2.0
- ❑ **2013** : 2.4.1
- ❑ **2017** ; dernière version 2.5.1

UML

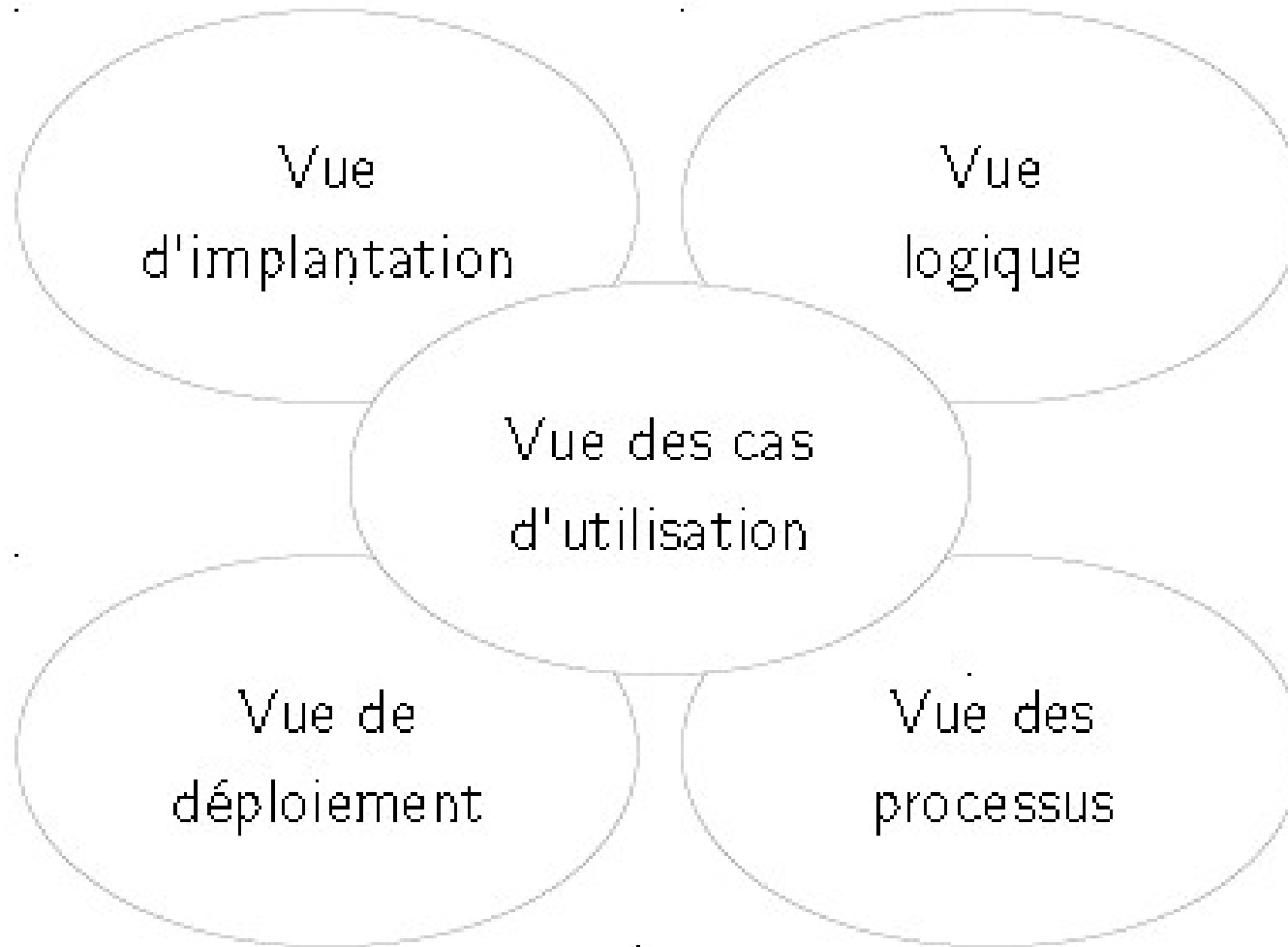
- Langage graphique : Ensemble de diagrammes permettant de modéliser le logiciel selon différentes vues et à différents niveaux d'abstraction
 - Modélisation orientée objet : modélisation du système comme un ensemble d'objets interagissant
- UML n'est pas une méthode de conception
- +UML est un outil indépendant de la méthode

Différents diagrammes UML

Représentation du logiciel à différents points de vue :

- **Vue des cas d'utilisation** : vue des acteurs (besoins attendus)
- **Vue logique** : vue de l'intérieur (satisfaction des besoins)
- **Vue d'implantation** : dépendances entre les modules
- **Vue des processus** : dynamique du système
- **Vue de déploiement** : organisation environnementale du logiciel

La vue « 4+1 » de ph. Kruchten



Diagrammes UML

Diagrammes structurels :

- Diagramme de classes
- Diagramme d'objets
- Diagramme de composants
- Diagramme de déploiement
- Diagramme de paquetages
- Diagramme de structure composite
- Diagramme de profils

Diagrammes comportementaux:

- Diagramme de cas d'utilisation
- Diagramme états-transitions
- Diagramme d'activité

Diagrammes d'interaction :

- Diagramme de séquence
- Diagramme de communication
- Diagramme global d'interaction
- Diagramme de temps

14 diagrammes hiérarchiquement dépendants
Modélisation à tous les niveaux le long du processus de développement

Exemple d'utilisation des Diagrammes

Spécification

- **Diagrammes de cas d'utilisation** : besoins des utilisateurs
- **Diagrammes de séquence** : scénarios d'interactions entre les utilisateurs et le logiciel, vu de l'extérieur
- **Diagrammes d'activité** : enchaînement d'actions représentant un comportement du logiciel

Conception

- **Diagrammes de classes** : structure interne du logiciel
- **Diagrammes d'objet** : état interne du logiciel à un instant donné
- **Diagrammes états-transitions** : évolution de l'état d'un objet
- **Diagrammes de séquence** : scénarios d'interactions avec les utilisateurs ou au sein du logiciel
- **Diagrammes de composants** : composants physiques du logiciel
- **Diagrammes de déploiement** : organisation matérielle du log